

UML Modellierung

Indooro – Indoor Navigation für Supermärkte

HTL – Softwareentwicklung
Lerneinheit (20 Minuten)

Team: Indooro

Was ist UML?

- Unified Modeling Language
- Standard zur Beschreibung von Software-Systemen
- Visuelle Kommunikation zwischen Entwicklern
- Unabhängig von Programmiersprache

Wozu verwendet man UML?

- Systeme planen bevor Code entsteht
- Komplexität reduzieren
- Team-Kommunikation
- Dokumentation
- Architektur verstehen

Grenzen von UML

- Kein Ersatz für Code
- Runtime-Verhalten nur begrenzt sichtbar
- Kann schnell zu detailliert werden
- Pflegeaufwand

Typische UML Fehler

- Klassen $\neq$ Objekte
- Zu viele Beziehungen
- Falsche Vererbung
- Diagramm ohne Use-Case Bezug

Unser Projekt: Indooro

Ziel:

Navigation innerhalb eines Supermarktes

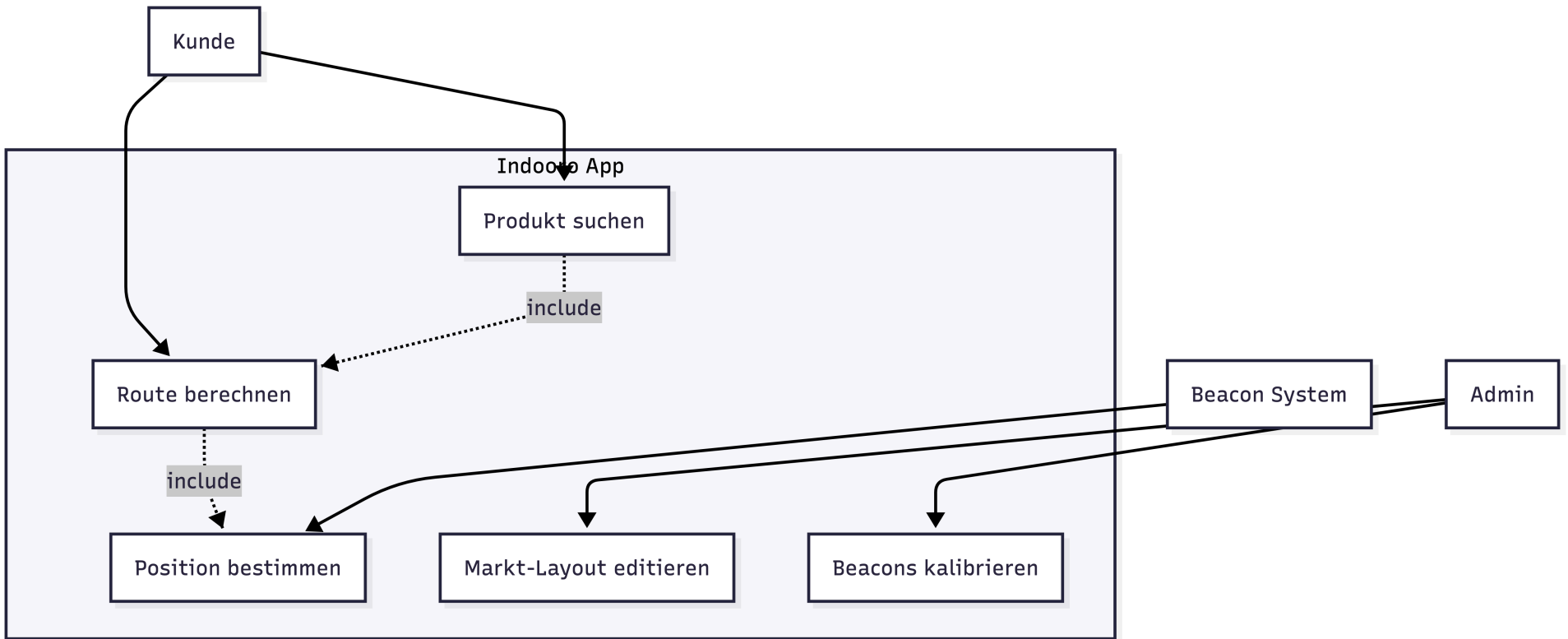
Ablauf:

- Kunde sucht Produkt
- Position wird bestimmt
- Route wird berechnet
- Produkt wird gefunden

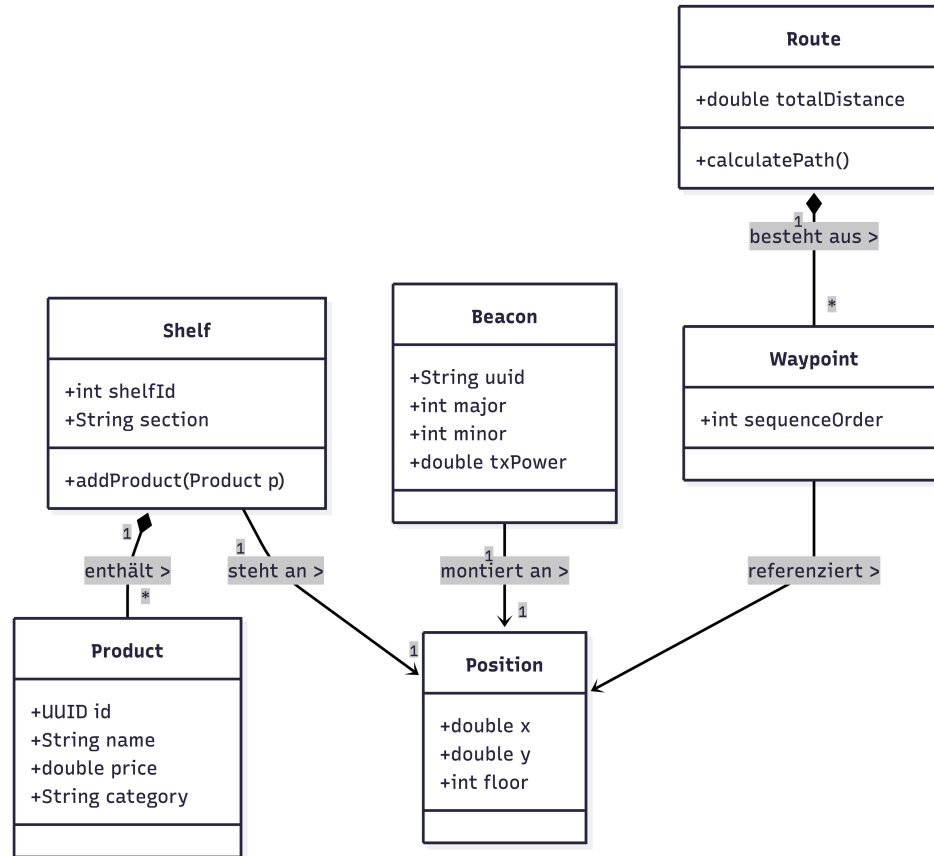
Warum UML bei Indooro?

- Backend + App abstimmen
- Datenmodell planen
- Navigationslogik verstehen
- Use Cases ableiten

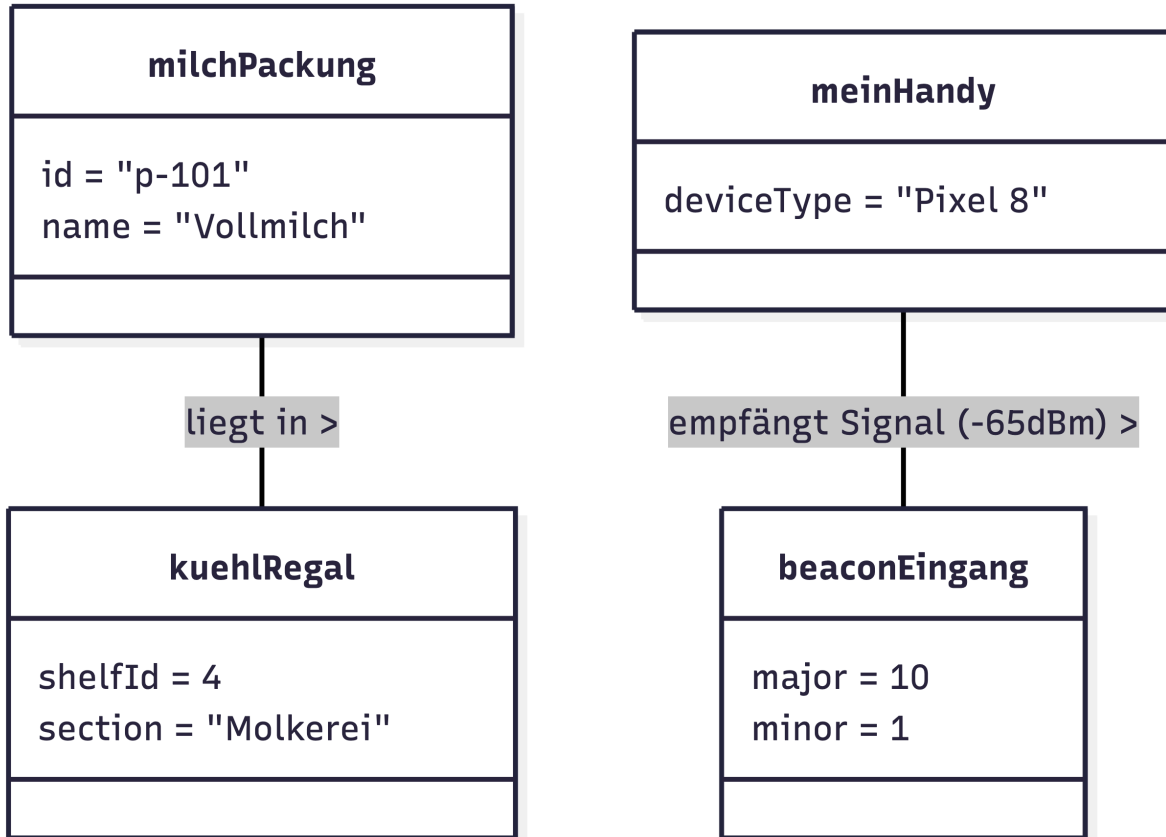
Use Case Diagramm



Klassendiagramm



Objektdiagramm



Traceability

Use Case 1: Produkt suchen → Klassen:

- **Product:** Speichert Name und ID der Ware.
- **Shelf:** Kennt die physische Position (x/y) im Markt.
- **SearchService:** Logik, die die Datenbank durchsucht.
- **AppUI:** Das Interface, wo der Kunde tippt.

Use Case 2: Route berechnen → Klassen:

- **Beacon:** Liefert Bluetooth-Signal zur Ortung.
- **UserPosition:** Speichert, wo der Kunde steht.
- **RouteCalculator:** Algorithmus für den Weg.
- **StoreMap:** Liefert Infos, wo Wände/Regale sind.

Projekt-Artefakte (Belege)

Vom UML zum echten System (Draft)

1. Java Klasse (aus dem Klassendiagramm)

```
public class Product {  
    public UUID id;  
    public String name;  
    public Shelf shelf; // Hier ist die Assoziation aus dem UML!  
}
```

2. Datenbank-Schema (Entwurf)

```
CREATE TABLE Product (  
    id VARCHAR(50) PRIMARY KEY,  
    name VARCHAR(100),
```

```
shelf_id INT REFERENCES Shelf(id)  
);
```


Diagramm Review (Fehlerkultur)

Typische Anfängerfehler am Beispiel Indooro

Fehler	Vorher ❌	Nachher ✅	Erklärung
1. Vererbung statt Assoziation	Beacon erbt von Position	Beacon hat eine Position	Ein Beacon <i>*ist*</i> keine Position. Er ist ein Gerät, das physisch dort montiert ist.
2. Objekte im Klassendiagramm	Klasse heißt "Milch"	Klasse heißt "Product"	Klassen sind abstrakte Baupläne. "Milch" ist nur ein konkretes Objekt zur Laufzeit.
3. Falsche Abhängigkeit	Product kennt Route	Route kennt Product	Die Milch im Regal weiß nicht, wer zu ihr navigiert.

KI-resistente Aufgaben

- Diagramm begründen lassen
- Live Modellierung
- Projektbezug erzwingen
- Review & Diskussion

KI – Pro & Contra

Pro

- Schnelle Diagrammerstellung
- Syntax Hilfe

Contra

- Logische Fehler
- Blindes Übernehmen

Zukunft von UML & KI

- Text → UML automatisch
- Skizze → Code
- KI als Assistenz
- Architektur bleibt menschlich

Verwendete Prompts

Ziel	Prompt (Komplett)	Tool	Output & Vertrauen
Komplette Struktur, Diagramme & Artefakte	Du bist ein erfahrener Software-Architekt und Dozent für Informatik an einer HTL (Höhere Technische Lehranstalt). Ich habe einen Arbeitsauftrag für die Schule, den ich am Beispiel meines aktuellen Projekts "Indooro" lösen muss. Bitte erarbeite den kompletten Inhalt für diese Präsentation (Dauer ca. 20 Minuten). ### 1. Der Projekt-Kontext (Indooro) Indooro ist ein Indoor-Navigationssystem für Supermärkte (SPAR). - **Backend:** Java 21 / Quarkus / OpenSearch. - **Frontend/Mobile:** Capacitor-App (Hybrid) mit Bluetooth-Zugriff. - **Hardware:** 5 iBKS USB Beacons im Laden für Trilateration. - **Daten:** JSON-Layout des Marktes und Produkt-Daten. ### 2. Das Szenario (Der Rote Faden)	ChatGPT / Gemini	Output: Komplettes Skript + HTML Struktur. Vertrauen: Mittel. Struktur super, Logik und Hintergrundwissen mussten wir uns selbst erarbeiten/prüfen.

Quellen & Links

- [OMG UML Specification](#)
- [draw.io](#)
- [PlantUML](#)
- [Mermaid](#)
- [Reveal.js](#)

